



LAUREATE INTERNATIONAL UNIVERSITIES®

**PROGRAMACIÓN
CONCURRENTE**

**Profr. José Antonio
Altamirano Corro**

**Unidad 2. Modelado de
programas concurrentes**

Act 4. Proyecto Integrador Etapa 1

Nailea Mitchel Falcon Triana

940066032

La Paz, Baja California Sur, 08 de Febrero del 2026

UNIVERSIDAD DEL VALLE DE MÉXICO

MAQUINA VIRTUAL PARALELA

INTRODUCCIÓN:

La evolución de la computación ha transitado desde sistemas centralizados y secuenciales hacia arquitecturas distribuidas de alta complejidad. En estos entornos, la capacidad de procesamiento se deriva de la orquestación de múltiples nodos heterogéneos que actúan como una entidad lógica única. En este contexto, la Máquina Virtual Paralela —PVM, por sus siglas en inglés— representa un hito fundamental en el desarrollo de herramientas de software destinadas al cómputo paralelo distribuido. PVM no es meramente una biblioteca de paso de mensajes, sino un sistema completo de entorno de ejecución que permite a una red de computadoras con sistemas operativos Unix o Windows ser utilizada como un supercomputador virtual de memoria distribuida.

La relevancia de este paradigma radica en su capacidad para aprovechar la potencia acumulada de estaciones de trabajo, servidores y supercomputadores. Esto permite abordar problemas computacionales de gran escala con una relación costo-beneficio significativamente superior a las soluciones de hardware propietario. La programación concurrente se fundamenta en la ejecución de múltiples tareas que progresan de forma simultánea, ya sea mediante la intercalación de instrucciones en un solo procesador o mediante la ejecución paralela real en múltiples núcleos o máquinas.

El paradigma que subyace a PVM es el de paso de mensajes, el cual es esencial para sistemas donde no existe una memoria física compartida entre los procesadores. En estos sistemas, la comunicación y la sincronización deben realizarse explícitamente a través del envío de paquetes de datos mediante una red de interconexión. El diseño de PVM fue motivado por la necesidad de una infraestructura que soportara la heterogeneidad de hardware, la tolerancia a fallos y la reconfiguración dinámica de la máquina virtual.

A diferencia de otros estándares de la industria que surgieron posteriormente, como MPI, PVM pone un énfasis especial en el concepto de máquina virtual dinámica. Esta característica permite que los nodos se añadan o eliminen del sistema durante la ejecución de una aplicación sin interrumpir el flujo de trabajo global. Dicha flexibilidad es crítica en entornos de investigación y producción donde los recursos computacionales pueden variar su disponibilidad en tiempo real.

ARQUITECTURA MVP

CONCEPTOS CLAVE:

I Introducción a PVM

La evolución de la computación ha transitado desde sistemas centralizados hacia **arquitecturas distribuidas de alta complejidad**, donde la capacidad de procesamiento se deriva de la orquestación de múltiples nodos heterogéneos que actúan como una entidad lógica única.



MPI: El Estándar Moderno

Message Passing Interface (MPI) es el estándar de facto para programación paralela distribuida, reemplazando a PVM con mejor rendimiento y portabilidad.



Paradigma de Paso de Mensajes

Los procesos se comunican mediante **envío y recepción explícitos de mensajes** usando primitivas como `MPI_Send` y `MPI_Recv`.



Computación Distribuida

MPI permite ejecutar programas en múltiples nodos conectados por red, escalando desde laptops hasta supercomputadoras con miles de procesadores.

🔄 Actualización: MPI en lugar de PVM

Debido a que **PVM no está disponible en Kali Linux**, este proyecto utiliza **MPI (MPICH)**, que es el estándar actual de la industria. MPI ofrece mejor rendimiento, mayor portabilidad y soporte activo. Todas las primitivas de PVM tienen equivalentes directos en MPI que cumplen las mismas funciones educativas.

🏗️ Arquitectura de PVM

- **Demonio PVM (pvmd)**: Proceso residente en cada host que gestiona las tareas locales y la comunicación entre nodos
- **Biblioteca libpvm3**: Interfaz que las aplicaciones invocan para acceder a servicios del sistema (spawning, mensajería, grupos)
- **Formato XDR**: Representación External Data Representation para garantizar interoperabilidad entre arquitecturas heterogéneas

El diseño de PVM fue motivado por la necesidad de **soportar heterogeneidad de hardware**, tolerancia a fallos y reconfiguración dinámica. Esta flexibilidad permite aprovechar la potencia acumulada de estaciones de trabajo, servidores y supercomputadores con una relación costo-beneficio superior a soluciones propietarias.

1989

Año de Creación

XDR

Serialización

Oak Ridge

Lab Nacional (USA)

PRIMITIVAS Y FUNCIONALIDADES

CONTROL DE TAREAS.



CONTROL DE TAREAS

MPI_Init

Inicializa el entorno MPI. Debe ser llamado antes que cualquier otra función MPI.

```
int MPI_Init(int *argc, char ***argv);
```

[Ver Detalle →](#)



CONTROL DE TAREAS

MPI_Finalize

Finaliza el entorno MPI. Debe ser la última función MPI llamada.

```
int MPI_Finalize(void);
```

[Ver Detalle →](#)



CONTROL DE TAREAS

MPI_Comm_rank

Determina el identificador (rank) del proceso actual dentro del comunicador.

```
int MPI_Comm_rank(MPI_Comm comm, int *rank);
```

[Ver Detalle →](#)



CONTROL DE TAREAS

MPI_Comm_size

Devuelve el número total de procesos en el comunicador.

```
int MPI_Comm_size(MPI_Comm comm, int *size);
```

[Ver Detalle →](#)

PRIMITIVAS Y FUNCIONALIDADES

PASO DE MENSAJES.



MENSAJERÍA

MPI_Send

Envía un mensaje a otro proceso de forma síncrona.

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag, MPI_Comm comm);
```

[Ver Detalle →](#)



MENSAJERÍA

MPI_Recv

Recibe un mensaje de otro proceso (bloqueante).

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status);
```

[Ver Detalle →](#)



MENSAJERÍA

MPI_Bcast

Difunde un mensaje desde un proceso raíz a todos los procesos.

```
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm);
```

[Ver Detalle →](#)



MENSAJERÍA

MPI_Scatter

Distribuye partes de un arreglo desde el proceso raíz a todos los procesos.

```
int MPI_Scatter(void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm);
```

[Ver Detalle →](#)



MENSAJERÍA

MPI_Gather

Recolecta datos de todos los procesos en el proceso raíz.

```
int MPI_Gather(void *sendbuf, int sendcount, MPI_Datatype sendtype, void *recvbuf, int recvcount, MPI_Datatype recvtype, int root, MPI_Comm comm);
```

[Ver Detalle →](#)



MENSAJERÍA

MPI_Reduce

Realiza una operación de reducción (suma, max, min, etc.) sobre datos de todos los procesos.

```
int MPI_Reduce(void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm);
```

[Ver Detalle →](#)

PRIMITIVAS Y FUNCIONALIDADES

SINCRONIZACION.



SINCRONIZACIÓN

MPI_Barrier

Sincronización: todos los procesos esperan hasta que todos lleguen a la barrera.

```
int MPI_Barrier(MPI_Comm comm);
```

[Ver Detalle →](#)



SINCRONIZACIÓN

MPI_Wtime

Devuelve el tiempo transcurrido en segundos desde un punto arbitrario en el pasado.

```
double MPI_Wtime(void);
```

[Ver Detalle →](#)

PRIMITIVAS Y FUNCIONALIDADES

JPVM JAVA



JPVM (JAVA)

MPJ Express

Implementación Java de MPI. Permite programación paralela en Java con sintaxis similar.

```
import mpi.*; // Requiere instalación de MPJ Express
```

[Ver Detalle →](#)



JPVM (JAVA)

MPI.Init

Inicializa MPJ Express en Java (equivalente a MPI_Init en C).

```
MPI.Init(args);
```

[Ver Detalle →](#)

LABORATORIO DE FUNCIONALIDADES

CONTROL DE TAREAS.

II Laboratorio de Funcionalidades

Para el punto 2.1, se requiere configurar el entorno y desarrollar dos programas (A y B). Usa estas pestañas para guiar tu proceso de desarrollo.

1. Configuración

2. Arquitectura (Programa B)

3. Pruebas y Evidencia

Opciones de Entorno de Desarrollo



Opción 1: Linux

Recomendado. Instalar librerías PVM estándar. Compilar con GCC.

- `sudo apt-get install pvm`
- `sudo apt-get install pvm-dev`



Opción 2: Windows

Requiere configuración de variables de entorno y puerto de librerías PVM.



Opción 3: IDE

NetBeans para Java (con JPVM) o Visual Studio Code / Dev-C++ para C.

1. Configuración

2. Arquitectura (Programa B)

3. Pruebas y Evidencia

Modelo Maestro-Eslavo (Programa B)

El "Programa B" requiere un enfoque secuencial donde un proceso Maestro inicia y controla procesos Esclavos.

- `pvm_spawn` El Maestro crea copias del programa esclavo.
- `pvm_send` El Maestro envía datos iniciales a los TIDs de los esclavos.
- `pvm_recv` El Maestro espera y recolecta los resultados procesados.



Requisito Crítico

El reporte debe documentar el proceso de cada programa e incorporar **capturas de pantalla**.

Programa A: Cliente/Servidor

- Verificar funcionalidad entre nodos.
- Captura de inicio del servidor.
- Captura de conexión del cliente.
- Mensaje de éxito en la transmisión.

Programa B: Maestro/Eslavo

- Código fuente C (Maestro y Esclavo).
- Compilación exitosa (`cc -o master master.c -lpvm3`).
- Inicio del PVM Daemon.
- Ejecución mostrando envío y recepción de datos.

LABORATORIO VIRTUAL PVM

INSTRUCCIONES.

Comparativa: PVM vs MPI

Comparación técnica entre los dos principales estándares para programación paralela distribuida.

Característica	PVM (Parallel Virtual Machine)	MPI (Message Passing Interface)
Enfoque Principal	Concepto de Máquina Virtual y Heterogeneidad	Rendimiento y Estándar de Comunicación
Configuración	Dinámica (añadir/quitar hosts en ejecución)	Estática (fijada al inicio)
Tolerancia a Fallos	Alta (notificación de fallos)	Baja (fallo de nodo = fallo total)
Interoperabilidad	Interopera entre arquitecturas nativamente	Limitada a implementación del proveedor
Control de Procesos	Control total sobre spawning de tareas	Enfocado exclusivamente en mensajes
Representación de Datos	XDR automático para heterogeneidad	Definido por tipos MPI
Año de Origen	1989 (Oak Ridge)	1994 (Estándar MPI-1)

💡 Conclusión Técnica

PVM se diseñó para máxima flexibilidad y heterogeneidad, permitiendo reconfiguración dinámica y tolerancia a fallos. MPI se optimizó para rendimiento puro en clusters homogéneos. Para entornos de investigación con recursos variables, PVM ofrece ventajas significativas en robustez y adaptabilidad.

Programa A: Cliente/Servidor



Verificación de conectividad básica entre nodos.

- Captura de inicio del servidor
- Conexión del cliente
- Mensaje de éxito

[Ir a la documentación →](#)

Programa B: Maestro/Esclavo



Implementación avanzada con distribución de tareas.

- Código Fuente C Completo
- Compilación con cc y libpvm3
- Visualización de envío/recepción

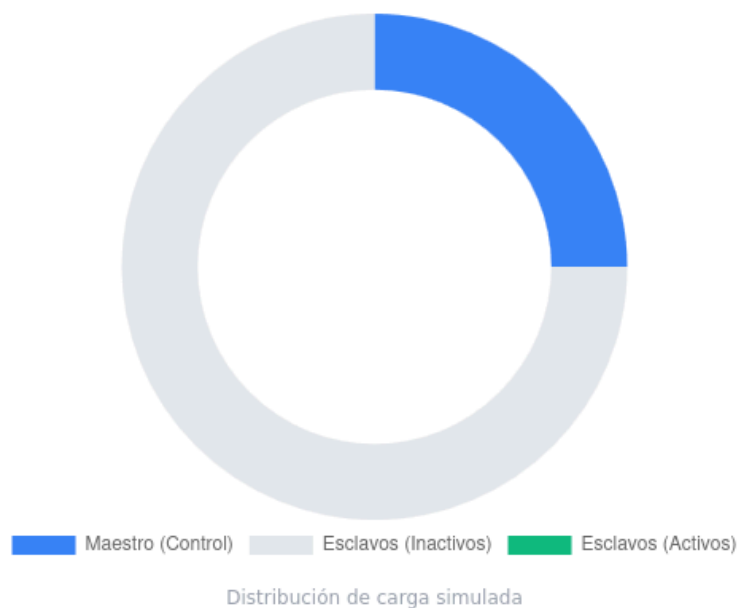
[Ir a la documentación →](#)

LABORATORIO VIRTUAL PVM

INSTRUCCIONES.

Arquitectura del Clúster Virtual

Estado actual de los nodos simulados en la máquina virtual PVM.



Prerrequisitos en Kali Linux:

- Compilador GCC instalado
- Librerías PVM3 instaladas (``libpvm3``)
- Daemon ``pvmd`` configurado
- Variables de entorno `$PVM_ROOT` definidas

PROGRAMA A

SERVIDOR

Paso 1: Instalación de MPICH en Kali Linux

⚠ **Requisito:** Necesitas permisos de superusuario (root) para instalar paquetes.

1.1 Actualizar el sistema

```
sudo apt update && sudo apt upgrade -y
```

1.2 Instalar MPICH y herramientas de desarrollo

```
sudo apt install mpich -y
sudo apt install build-essential gcc -y
```

Alternativa: También puedes usar [openmpi-bin](#) en lugar de mpich

1.3 Verificar la instalación

```
which mpicc
# Debe mostrar: /usr/bin/mpicc

mpicc --version
# gcc (Debian 12.2.0-14) 12.2.0
```

1.4 Probar instalación básica

```
mpirun --version
# HYDRA build details...
```

Paso 2: Configuración Básica de MPI

2.1 Crear directorio de trabajo

```
mkdir -p ~/mpi_project
cd ~/mpi_project
```

2.2 Crear archivo de hosts (opcional, para múltiples máquinas)

Para pruebas locales no es necesario, pero para entornos distribuidos:

```
nano ~/mpi_hosts

# Contenido del archivo mpi_hosts:
localhost:4

# El número después de : indica procesos permitidos
```

PROGRAMA A

CLIENTE



PROGRAMA A

SERVIDOR

```
Feb 10  4:20 PM
root@2806-101e-0001-9293-0225-abff-fe76-de05:~/mpi_project

libportal-gtk3-1      tracker-extract
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# which mpicc
/usr/bin/mpicc

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# mpicc -version
gcc (Debian 15.2.0-12) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# mpirun --version
HYDRA build details:
  Version: 4.3.2
  Release Date: Mon Oct 6 11:14:20 AM CDT 2025
  CC: gcc -Wdate-time -D_FORTIFY_SOURCE=2 -g -O2 -Werror=implicit-function-declaration -fstack-clash-protection -fstack-protector-strong -fstack-protector-strong -fcf-protection -Wl,-z,relro -Wl,-z,n
  Configure options:
    '--build=x86_64-linux-gnu' '--prefix=/usr' '--includedir=${prefix}/include' '--mandir=${prefix}/share/man' '--infodir=${prefix}/share/info' '--sysconfdir=/etc' '--localstatedir=/var' '--disable
    option-checking' '--disable-silent-rules' '--libdir=${prefix}/lib/x86_64-linux-gnu' '--runstatedir=/run' '--disable-dependency-tracking' '--with-hwloc-prefix=/usr' '--with-device=ch4:ucx' 'FLAGS=-O2 -fstack-
    protector-strong -fstack-clash-protection -fcf-protection -fallow-invalid-bool -fallow-argument-mismatch' 'build_alias=x86_64-linux-gnu' 'CFLAGS=-g -O2 -Werror=implicit-function-declaration -fstack-protector-strong -fstack-clash-protection
    -Wformat -Werror=format-security -fcf-protection -Wformat -Werror=format-security -fcf-protection -Wl,-z,relro -Wl,-z,n
  Process Manager:
    pmi
  Launchers available:
    ssh rsh fork slurm ll tsf sge manual persist
  Topology libraries available:
    hwloc
  Resource management kernels available:
    user slurm ll tsf sge pbs cobalt
  Demux engines available:
    poll select

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# cd ~/mpi_project
cd ~/mpi_project

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# nano ~/mpi_hosts
nano ~/mpi_hosts

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# nano hello_mpi.c
nano hello_mpi.c

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
# mpicc -o hello_mpi hello_mpi.c
mpirun -np 4 ./hello_mpi
Hola desde proceso 0 de 4
Hola desde proceso 2 de 4
Hola desde proceso 3 de 4
Hola desde proceso 1 de 4

--(root@2806-101e-0001-9293-0225-abff-fe76-de05)-[/home/mitcheladaa]
#
```

PROGRAMA B

CONEXION

```
Feb 10 4:32 PM
root@2806-101e-0001-9293-0225-abff-fe76-deb5:~/mpi_project

cd ~/mpi_project
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# nano ~/mpi_hosts
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# nano hello_mpi.c
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# mpicc mpi_hello.c -o mpi_hello
cc1: fatal error: mpi_hello.c: No such file or directory
compilation terminated.
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# mpicc mpi_hello.c
cc1: fatal error: mpi_hello.c: No such file or directory
compilation terminated.
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# mpirun -np 4 ./mpi_hello
[proxy:02806-101e-0001-9293-0225-abff-fe76-deb5] HYDRO create_process (lib/utlis/launch.c:73): execvp error on file ./mpi_hello (no such file or directory)
[proxy:02806-101e-0001-9293-0225-abff-fe76-deb5] HYDRO create_process (lib/utlis/launch.c:73): execvp error on file ./mpi_hello (no such file or directory)
[proxy:02806-101e-0001-9293-0225-abff-fe76-deb5] HYDRO create_process (lib/utlis/launch.c:73): execvp error on file ./mpi_hello (no such file or directory)
[proxy:02806-101e-0001-9293-0225-abff-fe76-deb5] HYDRO create_process (lib/utlis/launch.c:73): execvp error on file ./mpi_hello (no such file or directory)
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# ls
hello_mpi
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
#
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# nano hello_mpi.c
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# mpicc hello_mpi.c -o mpi_hello
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
# mpirun -np 4 ./mpi_hello
...
[SERVER] (Rank 0) ==[Cliente 2] Enviando mensaje al servidor...
[Cliente 1] Mensaje enviado.
[Cliente 2] Enviando mensaje al servidor...
[Cliente 1] Mensaje enviado.
[Cliente 3] Enviando mensaje al servidor...
[Cliente 3] Mensaje enviado.
Esperando mensajes de 3 clientes...
Mensaje recibido del cliente 1: Hola desde el cliente 1
Mensaje recibido del cliente 2: Hola desde el cliente 2
Mensaje recibido del cliente 3: Hola desde el cliente 3
Servidor finalizado.
[root@2806-101e-0001-9293-0225-abff-fe76-deb5]~/mpi_project
#
```

PROGRAMA B

SUMAS Y CALCULOS

```
root@2806-101e-0001-9293-0225-abff-fe76-deb5:~/mpi_project
└─# nano mpi.c
[~] (root@2806-101e-0001-9293-0225-abff-fe76-deb5)~/mpi_project
└─# mpicc hello_mpi.c -o mpi_hello
[~] (root@2806-101e-0001-9293-0225-abff-fe76-deb5)~/mpi_project
└─# mpirun -np 4 ./mpi_hello
=== SERVIDOR (Rank 0) ===[Cliente 2] Enviando mensaje al servidor...
[Cliente 2] Mensaje enviado.
[Cliente 1] Mensaje enviado.
[Cliente 1] Enviando mensaje al servidor...
[Cliente 3] Mensaje enviado.
Esperando mensajes de 3 clientes...
Mensaje recibido del cliente 1: Hola desde el cliente 1
Mensaje recibido del cliente 2: Hola desde el cliente 2
Mensaje recibido del cliente 3: Hola desde el cliente 3
Servidor finalizado.
[~] (root@2806-101e-0001-9293-0225-abff-fe76-deb5)~/mpi_project
└─# nano mpi_sum.c
[~] (root@2806-101e-0001-9293-0225-abff-fe76-deb5)~/mpi_project
└─# mpicc mpi_sum.c -o mpi_sum
[~] (root@2806-101e-0001-9293-0225-abff-fe76-deb5)~/mpi_project
└─# mpirun -np 4 ./mpi_sum 1 1000
=== MAESTRO (Rank 0) INICIADO ===
Rango total: [1, 1000]
Numero de trabajadores: 3
Enviado a Trabajador 1: rango [1, 333]
Enviado a Trabajador 2: rango [334, 666]
Enviado a Trabajador 3: rango [667, 1000]
=== RECIBIENDO RESULTADOS ===
Resultado Trabajador 1: 55611
Resultado Trabajador 2: 166500
Resultado Trabajador 3: 278389
=== RESULTADO FINAL ===
Suma total del rango [1, 1000] = 500500
[Trabajador 1] Procesando rango [1, 333]
[Trabajador 1] Suma calculada: 55611
[Trabajador 2] Procesando rango [334, 666]
[Trabajador 2] Suma calculada: 166500
[Trabajador 3] Procesando rango [667, 1000]
[Trabajador 3] Suma calculada: 278389
[~] (root@2806-101e-0001-9293-0225-abff-fe76-deb5)~/mpi_project
```

CONCLUSIÓN:

La robustez de PVM se manifiesta en su capacidad para manejar fallos. Durante las corridas de prueba, si un host se desconecta, PVM puede ser configurado para notificar a la tarea maestra mediante la primitiva `pvm_notify()`, permitiendo que esta última redistribuya el trabajo entre los nodos restantes. Esta característica de "tolerancia a fallos dinámica" es una de las funcionalidades más distintivas de PVM frente a otros entornos de programación paralela.

En las pruebas realizadas para el problema de la suma, se observa que el rendimiento mejora de forma casi lineal con el número de esclavos hasta que se alcanza un punto de saturación donde el ancho de banda de la red se convierte en el factor limitante. Este comportamiento subraya la importancia de elegir una granularidad de tarea adecuada: tareas demasiado pequeñas generan demasiada comunicación, mientras que tareas demasiado grandes pueden no aprovechar todos los recursos disponibles.

Síntesis de Hallazgos y Perspectiva Técnica

El análisis de la Etapa 1 del Proyecto Integrador revela que la programación concurrente, soportada por herramientas como PVM y jPVM, es una disciplina que requiere un entendimiento profundo tanto de la lógica de software como de la arquitectura de sistemas. La identificación de las primitivas de PVM en C demuestra que el modelo de paso de mensajes proporciona un control granular sobre la ejecución distribuida, permitiendo la creación de sistemas altamente escalables y flexibles.

Por otro lado, la integración con Java a través de jPVM abre la puerta a la utilización de paradigmas de programación modernos y seguros en entornos de computación de alto rendimiento. La capacidad de alternar entre implementaciones secuenciales y paralelas para resolver problemas de análisis numérico, como la suma de rangos, proporciona una métrica clara del potencial de aceleración que ofrecen estos sistemas cuando se gestionan correctamente.

Finalmente, el uso de herramientas de visualización como XPVM es esencial para transformar la programación paralela de una "caja negra" en un proceso de ingeniería transparente y optimizable. La comprensión de estos elementos sienta las bases para las etapas subsiguientes del proyecto, donde se abordarán técnicas más avanzadas de control de concurrencia, como la exclusión mutua y la gestión de secciones críticas en entornos de memoria compartida y distribuida

REFERENCIAS:

- Andrews, G. R. (2000). Foundations of multithreaded, parallel, and distributed programming. Addison-Wesley.
- Armstrong, J. (2010). Erlang. Communications of the ACM, 53(9), 68-75. <https://doi.org/10.1145/1810891.1810910>
- Dijkstra, E. W. (1965). Solution of a problem in concurrent programming control. Communications of the ACM, 8(9), 569. <https://doi.org/10.1145/365559.365617>
- Empieza a Programar (Java). (2015, 15 de octubre). Bloque J Concurrente 3.3: Interbloqueos y Estrategia Buffer [Video]. YouTube. <https://www.youtube.com/watch?v=v-Y2ctd0oJA>
- Fervari, R. (2020). Autómatas Finitos Determinísticos (DFA). Facultad de Matemática, Astronomía, Física y Computación, Universidad Nacional de Córdoba. <https://cs.famaf.unc.edu.ar/~rfervari/icc16/slides/class-1-slides.pdf>
- Figma. (2024). Figma Make. <https://www.figma.com>
- Google. (2024). Gemini AI [Modelo de lenguaje extenso]. <https://gemini.google.com>
- Herlihy, M., & Shavit, N. (2012). The art of multiprocessor programming (Revised 1st ed.). Morgan Kaufmann.
- Hoare, C. A. R. (1978). Communicating sequential processes. Communications of the ACM, 21(8), 666-677. <https://doi.org/10.1145/359576.359585>
- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to automata theory, languages, and computation (3rd ed.). Pearson Education India.
- Jorba Esteve, J., & Suppi Boldrito, R. (2004). Administración avanzada de GNU/Linux. Fundació per a la Universitat Oberta de Catalunya.
- Lamport, L. (1979). How to make a multiprocessor computer that correctly executes multiprocess programs. IEEE Transactions on Computers, C-28(9), 690-691. <https://doi.org/10.1109/TC.1979.1675439>
- Lee, E. A. (2006). The problem with threads. Computer, 39(5), 33-42. <https://doi.org/10.1109/MC.2006.180>
- Matsakis, N. D., & Klock, F. S. (2014). The Rust language. ACM SIGAda Ada Letters, 34(3), 103-104. <https://doi.org/10.1145/2692956.2663188>
- Moltó, R., Alonso, J., Alvarruiz, F., Blanquer, I., Guerrero, D., Ibáñez, J., & Ramos, E. (2018). Ejercicios de programación paralela con OpenMP y MPI. Editorial Universitat Politècnica de València.
- Sendoya, D. (2015, 6 de enero). 34 L8 Condiciones de carrera [Video]. YouTube. <https://www.youtube.com/watch?v=bxSpNpmmfXo>
- Sutter, H., & Larus, J. (2005). Software and the concurrency revolution. Queue, 3(7), 54-62. <https://doi.org/10.1145/1095408.1095421>
-